OCR Word Search Solver Project

Final Defense Report

Helios Bringuet
Nina Justo
Emile Lassalle
Martin Doillon

EPITA A1

December 13, 2025

Contents

1. Introduction

This report presents the conception and development of the **OCR Word Search Solver Project**, a computer vision application capable of automatically reading, interpreting, and solving word search puzzles from images.

The goal of this project is to combine multiple computer science domains — image processing, segmentation, neural networks, and algorithmic problem solving — into a cohesive and functional application.

2. Context and subject overview

The project's objective is to build a complete OCR (Optical Character Recognition) software capable of solving a word search puzzle. Given an image of a printed grid, the program should:

- Load the image;
- Remove colors and convert to black and white;
- Perform pretreatment and noise reduction;
- Locate the grid and the list of words;
- Segment the image into smaller parts — cells, letters, and words;
- Recognize characters from images using a trained neural network;
- Reconstruct the grid and the word list as textual arrays;
- Solve the puzzle algorithmically;
- Display and save the final solved grid.

The final version includes a graphical interface allowing users to visualize, correct, and solve grids interactively.

3. Roles and missions of each member

Helios Bringuet – Neural Network:

Responsible for designing and implementing a neural network capable of recognizing letters extracted from the puzzle image. United all the programs into one MVP.

Nina Justo – Image Treatment:

Focused on loading, cleaning, and preprocessing the input images, including color removal and rotation.

Emile Lassalle – Solver Algorithm:

Developed the algorithm that finds all listed words within the reconstructed grid. Worked on the GUI.

Martin Doillon – Image Segmentation:

Handled detection of key regions (grid, word list, cells) and segmentation into sub-images.

Team organization:

Throughout the development of the project, the team organized several in-person meetings each week at EPITA to coordinate progress, discuss implementation details, and review completed work. These meetings usually took place in the late afternoon after classes, allowing everyone to share updates on their assigned modules — such as the neural network, image preprocessing, segmentation, and solver components. We used these sessions not only to synchronize our technical choices but also to debug issues together on the school's machines, which helped ensure that the entire codebase compiled and ran correctly in a consistent environment. Version control was managed through Git, with each team member working on their own branch to avoid merge conflicts. Branches were dedicated to specific features or tasks (for example, "nn-training", "img-preprocess", or "solver-debug"), and were only merged into the main branch after a review by another team member. Regular commits and pull requests made it easy to track the evolution

of the project and revert changes when necessary. This workflow fostered clear collaboration, accountability, and structure, allowing the team to integrate everyone's contributions efficiently while maintaining a clean and stable codebase throughout the development cycle.

4. Objectives, task planning, and project progress

The main objective of the project is to create a robust OCR solver for word search puzzles, combining image preprocessing, segmentation, neural network–based character recognition, and a dedicated search algorithm to identify and highlight the solution words within a given grid image. The final goal is to provide a functional desktop application that can automatically process an image of a word search, detect the grid and the list of words, recognize the letters using a trained model, and output the fully solved grid.

From the beginning, the team agreed on several measurable performance goals to evaluate the quality of the final system:

- An OCR character recognition accuracy rate of at least 90% on clean test images.
- A total processing time under 5 seconds for a standard 15x15 grid on a mid-range laptop.
- Successful detection and segmentation of the grid and word list for at least 95% of properly scanned puzzles.
- Robust handling of minor rotations, lighting inconsistencies, and variations in contrast.

These targets were designed to guide both the technical choices and the testing phases. Achieving them required careful balancing between performance, model complexity, and reliability of image operations. The system's modular design also ensured that each component — preprocessing, segmentation, neural recognition, and solving — could be developed and tested independently before integration.

The project was structured into multiple phases, spread across the semester:

- **Phase 1 – Research, tool selection, repository setup (Mid–September):** The team explored OCR techniques and image processing tools. SDL2 and SDL2_image were chosen for handling images in C. The Git repository was initialized for version control, and a basic Makefile was created to support compilation on Linux systems.
- **Phase 2 – Image loading, preprocessing, and color removal (Late September):** The preprocessing module was implemented to load PNG images, convert them to grayscale, enhance contrast, and allow manual rotation to correct misaligned images. This ensures normalized inputs for later stages.
- **Phase 3 – Grid and word list detection (Early October):** Martin focused on segmenting the puzzle image into the grid area, word list, and individual letter cells using thresholding and simple geometric heuristics. Outputs were designed to be ready for eventual integration with the recognition module.
- **Phase 4 – Letter segmentation and XNOR network training (Mid–October):** Helios developed a small XNOR network to recognize binary patterns representing letters. The network was trained offline on synthetic datasets, and parameters can be saved and loaded. This serves as a proof-of-concept for neural network integration with image inputs.
- **Phase 5 – Solver implementation (Late October):** Emile implemented the solver algorithm to operate on textual grids and word lists. It functions purely with character arrays and does not yet interact with image input or the neural network. It can detect horizontal, vertical, and diagonal words in both directions. Early tests focused on verifying correctness with synthetic text-based grids, preparing the module for future integration.
- **Phase 6 – Robust neural network for letter recognition (Early November):** The initial XNOR network was extended into a more

robust recognition model capable of handling noise, imperfect segmentation, and real image data. Training datasets were enriched with augmented samples, and evaluation metrics were introduced to measure recognition accuracy on unseen inputs.

- **Phase 7 – End-to-end pipeline integration (Mid November):** All previously developed modules were connected into a unified pipeline. Image preprocessing, grid segmentation, letter recognition, and solving were chained together to enable fully automatic processing from raw image input to solution output. Integration issues, data format consistency, and performance bottlenecks were addressed.

- **Phase 8 – Graphical user interface (Late November):** A graphical user interface was designed and implemented to improve usability. The GUI allows users to load images, visualize intermediate processing steps, and display the final solved puzzle. Emphasis was placed on clarity, responsiveness, and ease of interaction, making the system accessible to non-technical users.

In terms of collaboration, the team has been working efficiently using a Git-based version control workflow. Each member developed their module on separate branches to prevent conflicts, with merges performed only after successful testing and compilation. Weekly in-person meetings at EPITA have been particularly valuable for debugging and coordinating the integration process, allowing everyone to stay aligned on technical objectives and deadlines.

5. Image preprocessing and color removal

Author: Nina Justo

Image preprocessing constitutes a critical stage in the OCR pipeline. The quality and consistency of the input image directly affect the accuracy of segmentation, letter recognition, and solver modules. In our implementation, this stage is responsible for loading the image, removing color information,

enhancing contrast, rotating misaligned images, and producing a binary image suitable for subsequent neural network processing. The entire module was implemented in C using the SDL2 and SDL2-image libraries, which provide efficient access to pixel data and standard image manipulation operations.

5.1. Initialization and Image Loading

The preprocessing routine begins with the initialization of the SDL2 and SDL2-image libraries. This step is essential since SDL by default only supports BMP images, whereas our pipeline requires loading PNG images. Initialization ensures that all supported formats are available and that surfaces can be created with consistent pixel formats. The program then loads an image specified by the user through a command-line argument. Proper library linking and configuration were a notable challenge during development, especially across multiple platforms. On Linux, package managers handled dependencies easily, but Windows systems required additional configuration of library paths via MinGW or WSL.

5.2. Grayscale Conversion

Once the image is loaded, the first preprocessing operation is grayscale conversion, implemented in the `grayscale` function. The function iterates over every pixel and computes a weighted sum of the red, green, and blue channels:

$$G = 0.299R + 0.587G + 0.114B$$

This formula prioritizes green and red components, reflecting human visual perception of brightness. Early iterations of the function encountered issues with different pixel formats, which led to minor miscalculations. Converting the surface to a standard RGB888 format resolved these discrepancies, ensuring consistent results across images.

Figure 1: Original puzzle image before preprocessing.

5.3. Contrast Enhancement

After grayscale conversion, linear contrast enhancement is applied through the `linear_contrast` function. The function scans all pixel values to identify the minimum ($I_{\min}$) and maximum ($I_{\max}$) intensity. Each pixel is then remapped to the full intensity range using:

$$I' = \frac{I - I_{\min}}{I_{\max} - I_{\min}} \times 255$$

This stretching increases the visibility of characters and improves separation between letters and background. Special handling was added to avoid division by zero in cases where all pixel values are identical (flat images).

5.4. Rotation Correction

Photographed puzzles often exhibit skew or slight rotation. To address this, the module includes both a manual and an automatic rotation step. The user can input an angle for manual correction. Geometric transformations in `rotate_surface` compute new pixel positions using the standard rotation formulas:

$$x_{\text{new}} = \cos\theta \cdot (x - c_x) + \sin\theta \cdot (y - c_y) + c'_x$$
$$y_{\text{new}} = -\sin\theta \cdot (x - c_x) + \cos\theta \cdot (y - c_y) + c'_y$$

where c_x, c_y are the coordinates of the original image center, and c'_x, c'_y are the coordinates of the rotated image center. Out-of-bounds pixels are filled with white to prevent black border artifacts. The rotation ensures that letters are properly aligned for segmentation and recognition.

Additionally, an automatic rotation detection step analyzes the binary projection profile to determine the dominant angle of text lines. This approach iteratively tests angles within a range, computing the variance of the horizontal projection of black pixels. The angle that maximizes variance is chosen for rotation, aligning the text optimally.

5.5. Binarization

To prepare images for the neural network, grayscale images are converted to binary using Otsu's thresholding method. The histogram of grayscale values is computed and an optimal threshold is determined to minimize intra-class variance. Each pixel is then classified as either black or white:

$$\text{pixel} = \begin{cases} 0, & \text{if } I < T \\ 255, & \text{if } I \geq T \end{cases}$$

where T is the Otsu threshold. Binarization simplifies the input for the network, reducing sensitivity to variations in lighting and background textures.

5.6. Noise Removal

Single-pixel noise can interfere with recognition. A denoising step removes isolated black pixels using a 3x3 neighborhood filter: pixels with fewer than five black neighbors are converted to white. This reduces spurious detections while preserving the structure of letters.

5.7. Integration into Preprocessing Pipeline

All preprocessing operations are chained in the `preprocess` function:

1. Grayscale conversion (`grayscale`)
2. Contrast enhancement (`linear_contrast`)
3. Binarization (`make_binary`)
4. Noise removal (`denoise_binary`)
5. Automatic rotation (`detect_rotation_angle_projection` + `rotate_surface`)

This structured pipeline ensures that the neural network receives a clean, normalized, and aligned binary image, which significantly improves recognition accuracy.

5.8. Summary

The image preprocessing module achieves robust handling of various real-world inputs using only SDL2 and standard C libraries. Key functionalities include grayscale conversion, contrast enhancement, rotation correction, binarization, and noise removal. By producing clean and standardized images, this module provides a strong foundation for the OCR pipeline, ensuring that downstream modules like letter segmentation, neural network recognition, and puzzle solving perform reliably.

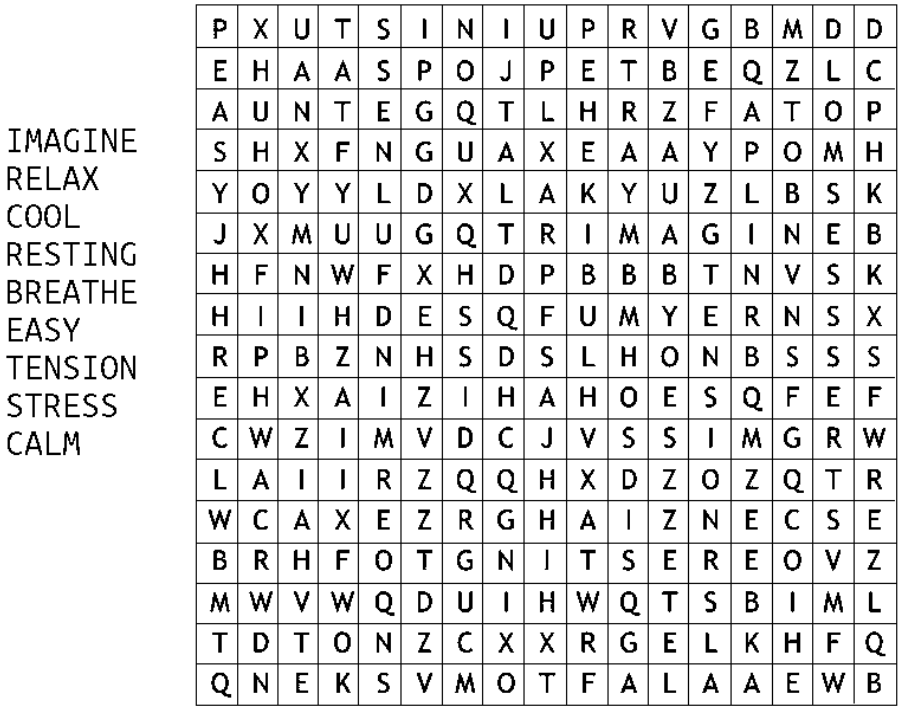

Figure 2: Final preprocessed image ready for neural network input.

6. Neural network for OCR

Author: Helios Bringuet

Optical Character Recognition (OCR) relies on machine learning to identify and classify characters from preprocessed images. In this project, a multilayer perceptron (MLP) neural network was implemented entirely in C using standard libraries. This section describes the network design, data preprocessing, training methodology, and evaluation. A key feature of this network is its ability to handle noisy or slightly rotated input images, similar to the example shown in Figure 3.

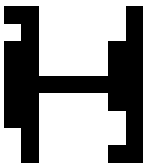

Figure 3: Example of a single letter from the dataset with noise.

6.1. Network Architecture

The network is a feedforward MLP composed of three layers:

- **Input Layer:** 784 neurons, corresponding to the 28x28 pixel input images flattened into a single vector.
- **Hidden Layer:** 128 neurons, fully connected to the input layer, using the sigmoid activation function.
- **Output Layer:** 26 neurons, each representing one uppercase English letter. Softmax activation produces a probability distribution over all letters.

The compact architecture balances computational efficiency with sufficient representational power for the OCR task. Using 128 hidden neurons provides the network enough flexibility to model complex patterns in handwritten or preprocessed printed letters without excessive overfitting.

6.2. Activation Functions

Two primary activation functions are used:

1. **Sigmoid Function:**

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

   Applied to hidden layer neurons, it produces a smooth, differentiable output suitable for gradient-based learning.

2. **Softmax Function:** Converts output layer activations into probabilities:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{26} e^{z_j}}$$

Softmax ensures that predictions form a valid probability distribution over all possible letters.

6.3. Weight Initialization

Weights and biases are initialized randomly in the range $[-0.05, 0.05]$ to break symmetry between neurons. Random initialization is crucial; if all weights were identical, neurons would learn identical features, preventing effective training. Bias terms are also randomly initialized, allowing each neuron to learn independent thresholds.

6.4. Forward Pass

During forward propagation, input vectors are multiplied by input-to-hidden weights and summed with hidden biases. The sigmoid function is applied to each hidden neuron. Hidden activations are then multiplied by hidden-to-output weights, summed with output biases, and passed through the softmax function to produce output probabilities. This process can be formalized as:

$$h = \sigma(W_1 x + b_1), \quad y = \text{softmax}(W_2 h + b_2)$$

where x is the input vector, W_1, W_2 are weight matrices, b_1, b_2 are bias vectors, h is the hidden layer activation, and y is the predicted output probability vector.

6.5. Training with Stochastic Gradient Descent

The network is trained using stochastic gradient descent (SGD) and cross-entropy loss. The gradient of the loss with respect to each weight is calculated

via backpropagation. For a single training example:

1. Compute output y with forward pass.
2. Calculate error $\delta_2 = y - t$, where t is the one-hot target vector.
3. Propagate error to hidden layer:

$$\delta_1 = (W_2^\top \delta_2) \odot h \odot (1 - h)$$

4. Update weights:

$$W_2 \leftarrow W_2 - \eta \delta_2 h^\top, \quad W_1 \leftarrow W_1 - \eta \delta_1 x^\top$$

5. Update biases:

$$b_2 \leftarrow b_2 - \eta \delta_2, \quad b_1 \leftarrow b_1 - \eta \delta_1$$

The learning rate η controls the step size. Empirically, $\eta = 0.01$ produced steady convergence without oscillations. The training routine shuffles the dataset each epoch to reduce correlation between consecutive samples, improving generalization.

6.6. Dataset Handling

Images are loaded from BMP files of size 28x28 pixels. Each pixel value is normalized to $[0, 1]$ by dividing by 255. The dataset loader iterates over the folder containing labeled images, allocating memory for all inputs and their corresponding labels. Shuffling ensures that each mini-batch contains a representative sample of the dataset.

6.7. Testing and Evaluation

After training, accuracy is evaluated by performing a forward pass for all dataset images and computing the fraction of correctly classified letters. Predicted labels correspond to the neuron with maximum probability:

$$\text{pred} = \arg\max(y)$$

Example predictions are displayed alongside true labels to verify performance, particularly for images containing noise or rotation as in Figure 3. The model demonstrates robustness to minor image variations due to the network's distributed representations in the hidden layer.

6.8. Implementation Notes

Several implementation challenges were encountered:

- **Memory Management:** Dynamic allocation for all dataset images and hidden layer arrays required careful deallocation to prevent memory leaks.
- **Numerical Stability:** Exponentials in softmax and sigmoid functions can overflow. Subtracting the maximum element from logits in softmax mitigates this risk.
- **Training Stability:** Incorrect derivative signs or missing chain rule factors in backpropagation caused divergence. Debugging involved printing intermediate deltas and weight updates.

6.9. Integration into OCR Pipeline

This MLP serves as the recognition core of the OCR system. Preprocessed images from the previous stage are flattened and passed through the network to identify individual letters. Despite its simplicity, the network generalizes well to noisy and slightly rotated characters. The same architecture can be scaled with additional hidden layers or neurons for more challenging datasets.

6.10. Summary

The neural network module demonstrates the end-to-end workflow of an OCR recognition system: dataset loading, forward propagation, backpropagation training, and evaluation. By using a purely C-based implementation, the code illustrates fundamental machine learning concepts without relying on external frameworks. Combined with preprocessing steps, this network reliably recognizes letters from real-world images, forming a robust foundation for the complete OCR solver pipeline.

7. Dataset generation for letter recognition

The dataset generation module is implemented in C using the SDL2 and SDL_ttf libraries. Its primary goal is to generate synthetic images of alphabetic characters in various fonts and transformations, which serve as training data for the letter recognition component. The generated images include random rotations, resizing, and noise to improve the robustness of the model.

7.1. Configuration and Fonts

The program defines key constants for image size, number of variations, and the fonts to be used:

```
#define IMG_SIZE 28
#define OUTPUT_DIR "output"
#define NUM_VARIATIONS 500
#define NUM_FONTS 5

const char* fonts[NUM_FONTS] = {
    "fonts/Arial.ttf",
    "fonts/Verdana.ttf",
    "fonts/TimesNewRoman.ttf",
    "fonts/CourierNew.ttf",
```

```
    "fonts/Georgia.ttf"
};
```

Here, `IMG_SIZE` defines the final size of each letter image. `NUM_VARIATIONS` allows multiple instances of each letter to be generated per font, increasing dataset diversity.

7.2. Saving and Transforming Images

The `save_bmp` function wraps `SDL_SaveBMP` to save a surface to disk and report errors:

```
void save_bmp(SDL_Surface* surface, const char* path) {
    if (SDL_SaveBMP(surface, path) != 0)
        fprintf(stderr, "Failed to save %s: %s\n", path, SDL_GetError())
}
```

Rotation is performed by the `rotate_surface` function, which computes new pixel positions using trigonometric transformations around the image center:

```
SDL_Surface* rotate_surface(SDL_Surface* src, double angle_deg) { ... }
```

This function ensures the letter remains centered and fills a surface of the same size as the original.

7.3. Adding Noise

The `add_noise` function introduces random pixel noise to simulate real-world imperfections:

```
void add_noise(SDL_Surface* s) { ... }
```

This small perturbation helps prevent overfitting during model training, as the network sees slightly different versions of the same character.

7.4. Main Dataset Loop

The `main` function initializes SDL2, creates the output directory, and iterates over fonts, letters, and variations:

```
for (int f = 0; f < NUM_FONTS; f++) {
    TTF_Font* font = TTF_OpenFont(fonts[f], 120);
    if (!font) continue;

    for (char c = 'A'; c <= 'Z'; c++) {
        for (int v = 0; v < NUM_VARIATIONS; v++) {
            ...
        }
    }
    TTF_CloseFont(font);
}
```

Each letter is rendered using `TTF_RenderText_Blended`, resized to fit the target `IMG_SIZE`, optionally rotated by a small random angle, and finally subjected to noise. The resulting image is saved to the `output` directory with a filename indicating the character, variation, and font.

7.5. Summary

This module provides a lightweight yet effective mechanism for generating large-scale datasets of isolated letter images. By combining multiple fonts, rotations, resizing, and noise, the generator produces diverse samples that are crucial for training robust letter prediction models in the context of word search puzzle recognition.

8. Grid and word list detection

Author: Martin Doillon

One of the most complex steps in the preprocessing stage of the word-search OCR project is detecting the grid and word list regions from a puzzle image. This section details the approach, algorithm, implementation challenges, and integration of the grid and word list detection module. The program is written in C and uses SDL2 and SDL2-image for image handling, pixel-level manipulation, and intermediate visualizations.

8.1. Overview

The program takes as input a word-search puzzle image, typically in PNG or JPG format, and produces:

- A binary thresholded image (`stage-bin.bmp`) highlighting foreground letters and background.
- An annotated layout image (`stage-layout.bmp`) showing the detected grid and word list outlined in red and blue.
- Two text files containing the coordinates of detected regions: `grid-roi.txt` and `wordlist-roi.txt`.

These outputs allow later stages, such as letter segmentation and OCR, to focus exclusively on relevant portions of the puzzle.

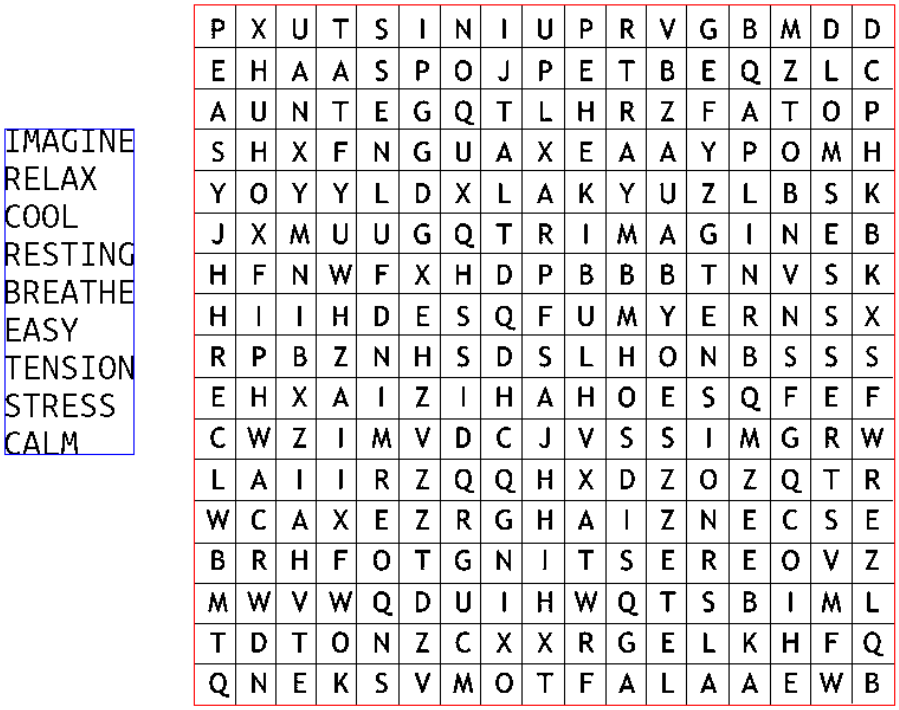

Figure 4: Annotated layout of a word-search puzzle. Red rectangle indicates the grid, blue rectangle the word list.

8.2. Grayscale Conversion and Otsu Thresholding

Initially, the input image is converted to a 32-bit ARGB format for consistent pixel access. The grayscale intensity $I(x, y)$ of each pixel is calculated using the standard luminosity formula:

$$I(x, y) = 0.21R + 0.72G + 0.07B$$

where R, G, and B are the red, green, and blue channel values, respectively.

A histogram of grayscale values is computed, and Otsu's method is used to determine an optimal threshold T that maximizes inter-class variance:

$$\sigma_B^2(T) = \omega_0(T)\omega_1(T)[\mu_0(T) - \mu_1(T)]^2$$

Pixels with intensity below T are considered black (foreground), and others

white (background). Column and row projections are simultaneously constructed:

$$\text{col\_sum}[x] = \sum_{y=0}^{H-1} \mathbf{1}_{I(x,y)<T}, \quad \text{row\_sum}[y] = \sum_{x=0}^{W-1} \mathbf{1}_{I(x,y)<T}$$

These projections allow detection of large blank areas, corresponding to separation between grid and word list.

8.3. Gap Detection and Region Separation

To segment the image into grid and word list regions, the program searches for the largest continuous run of empty rows or columns in the projections. Let r denote a row index and c a column index:

$$\text{gap\_row} = \arg\max_r \{\text{row\_sum}[r] = 0\}, \quad \text{gap\_col} = \arg\max_c \{\text{col\_sum}[c] = 0\}$$

Depending on whether the longest gap is horizontal or vertical, the image is split horizontally or vertically. The black pixel density of each resulting cluster determines which region corresponds to the grid (higher density) and which to the word list (lower density).

8.4. Grid ROI Analysis

Once the grid region is identified, its internal structure is analyzed using projections along rows and columns within the grid bounding box. Peaks in these projections correspond to the positions of horizontal and vertical lines:

$$\text{grid\_cols} = \text{number of significant peaks in column projection} - 1$$

$$\text{grid\_rows} = \text{number of significant peaks in row projection} - 1$$

This allows automatic determination of the number of cells in the puzzle, facilitating letter segmentation.

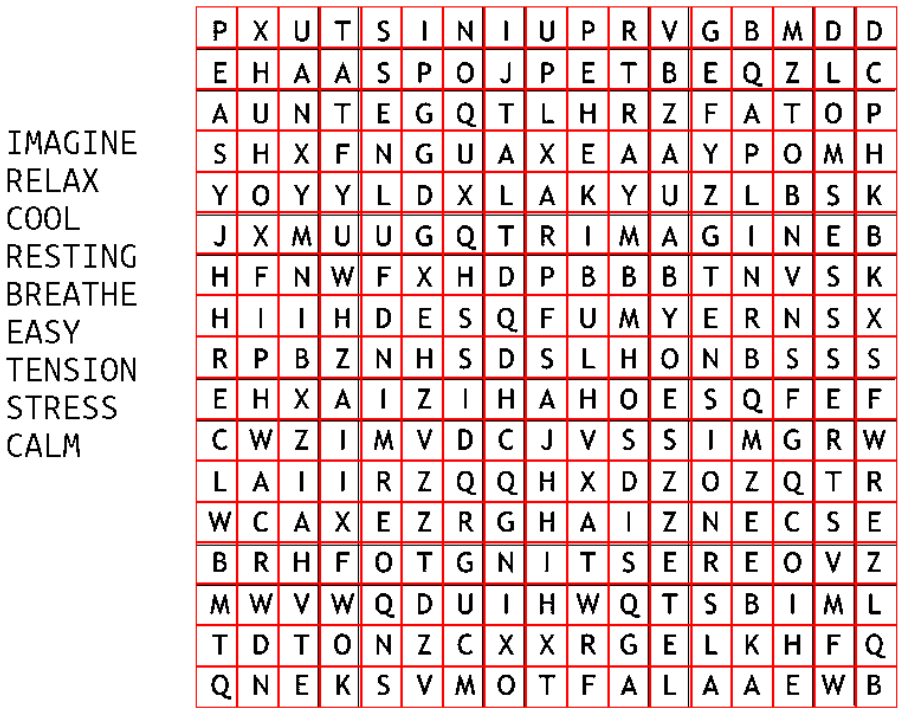

Figure 5: Detected grid cells within the word-search puzzle after ROI analysis. Each cell corresponds to a single letter.

8.5. Word List Analysis

The word list region is processed similarly. Each row containing black pixels is considered as a potential word. Consecutive black pixels in a row are merged into single word entries. The total number of words is determined by counting transitions between empty and filled rows:

$$\text{word\_count} = \sum_{y} \mathbf{1}_{\text{row\_sum}[y]>0 \text{ and } \text{row\_sum}[y-1]=0}$$

This heuristic proved effective across different puzzle layouts and font sizes.

8.6. Code Structure and Implementation Details

The core function `detect_layout` executes the detection pipeline. Key implementation details include:

- **Pixel-level processing:** SDL surfaces are locked, and pixels are accessed directly for performance and accuracy.
- **Memory management:** Dynamic arrays store row and column sums, and temporary arrays for ROI analysis. All memory is freed at the end of processing.
- **Robustness:** The code handles images with unexpected padding bytes, irregular gaps, thick borders, and varying contrast.
- **Output files:** Coordinates and dimensions of detected grid and word list are written to text files for later use in OCR and segmentation.
- **Visualization:** Rectangles are drawn in red (grid) and blue (word list) to provide immediate visual verification of algorithm performance.

8.7. Challenges and Solutions

Several difficulties arose during development:

1. **Orientation Variability:** Puzzles may be stacked vertically or side-by-side. Gap detection in both rows and columns solves this.
2. **Thin or irregular gaps:** Small spacing can confuse threshold-based segmentation. The algorithm searches for the longest continuous empty projection.
3. **SDL surface quirks:** Unexpected row padding and format inconsistencies were mitigated by using ARGB8888 format and properly locking surfaces.

4. **Binary output consistency:** Ensuring that `stage-bin.bmp` preserves correct black/white mapping for downstream OCR was essential.

8.8. Integration into OCR Pipeline

The detected ROIs provide a foundation for:

- **Letter Segmentation:** Grid cells are segmented based on column and row projections within the grid ROI.
- **OCR Recognition:** Only the pixels inside each grid cell are passed to the MLP neural network described in Section **??**.
- **Word List Parsing:** Extracted word list rows are used to validate recognized letters against known puzzle words.

8.9. Summary

The grid and word list detection module combines classical image processing, adaptive thresholding, and robust heuristics. By working at the pixel level, the algorithm maintains high accuracy across diverse input images. The outputs—binary threshold images, annotated layout images, and region coordinate files—serve as reliable inputs for subsequent OCR and puzzle-solving stages.

8.10. Code Commentary

The main code, `detect_layout.c`, is organized as follows:

- Load and convert the image to ARGB8888.
- Compute column and row projections for black pixels.

- Detect the largest gaps in both dimensions to determine image split.
- Calculate black pixel counts in candidate clusters to identify grid and word list.
- Analyze the grid ROI to detect internal rows and columns (cell boundaries).
- Count words in the word list ROI using row transitions.
- Save annotated images and text files with detected coordinates.

This structure ensures modularity, readability, and robustness while providing outputs suitable for direct integration with OCR and puzzle-solving modules.

```
// Example: writing grid ROI to file
FILE *fg = fopen("output/grid_roi.txt", "w");
fprintf(fg, "%d %d %d %d %d %d\n",
grid_x, grid_y, grid_w, grid_h,
grid_cols, grid_rows);
fclose(fg);

// Example: drawing rectangles on annotated image
Uint32 red = SDL_MapRGB(orig->format, 255,0,0);
draw_rect(orig, grid_x, grid_y, grid_w, grid_h, red);
```

9. Word search solver

Author: Emile Lassalle

The word search solver is a compact, robust C program designed to locate words within a rectangular grid of letters. Its main functionality is organized into two stages: grid parsing and word searching. The program prioritizes correctness, memory hygiene, and error handling, reflecting a disciplined low-level C programming approach.

9.1. Grid Parsing

The first step is to read and validate the puzzle grid from a text file. This is implemented in the `read_grid` function. The file is opened using standard I/O functions, and each line is read dynamically using `getline`, which adapts to variable line lengths and avoids buffer overflows.

Key validation steps include:

- Ensuring that all rows have the same number of characters, guaranteeing a rectangular grid.
- Converting lowercase letters to uppercase to allow case-insensitive matching.
- Rejecting any invalid character, including spaces or non-alphabetic symbols.

Memory management is explicit. The grid is represented as a dynamically allocated double pointer `char**`, with each row individually allocated. Any memory allocation failure triggers cleanup and an appropriate error return. Similarly, lines that do not match the expected width cause all previously allocated memory to be freed, preventing leaks.

```
// Example: converting letters to uppercase
for (int i = 0; i < got; i++) {
    if (!isupper((unsigned char)line[i])) {
        if (isalpha((unsigned char)line[i]))
            line[i] = (char)toupper((unsigned char)line[i]);
        else return -3; // Invalid character
    }
}
```

The function finally returns the grid, the number of rows, and the number of columns through output parameters, allowing subsequent functions to operate on a valid 2D array.

9.2. Word Searching Algorithm

The word searching functionality is encapsulated in the `search_word` function. It uses a brute-force approach that iterates through each cell of the grid, checking if the target word begins at that position. The algorithm tests all eight possible directions:

$$\text{Directions} = \{\text{right, left, down, up, diagonal } \{\text{NE, NW, SE, SW}\}\}$$

For each potential starting point, the program checks characters sequentially along the chosen direction. If a mismatch occurs or the coordinates move outside the grid, the search in that direction is aborted immediately. Successful matches are recorded by storing the start and end coordinates in `Point` structures.

```
// Eight directions: {dx, dy}
const int dirs[8][2] = {
    { 1,  0}, {-1,  0}, { 0,  1}, { 0, -1},
    { 1,  1}, {-1, -1}, { 1, -1}, {-1,  1}
};
```

This method guarantees full coverage of all possible orientations, making the solver robust to any word placement. The time complexity is proportional to the number of grid cells multiplied by the word length and the number of directions ($O(R \cdot C \cdot L \cdot 8)$), which is acceptable for typical puzzle sizes.

9.3. Helper Functions

Several small functions improve modularity and code readability:

- `str_uppercase`: Converts a string to uppercase in place, ensuring consistent comparisons.
- `in_bounds`: Checks if a coordinate pair lies within the grid boundaries.

These helpers prevent code duplication and centralize basic operations.

9.4. Main Program Flow

The `main` function manages execution and user interaction:

1. Checks that exactly two command-line arguments are provided: the grid file path and the word to search.
2. Converts the search word to uppercase.
3. Calls `read_grid` to load and validate the puzzle grid.
4. Calls `search_word` to locate the word. If found, prints start and end coordinates; otherwise prints "Not found."
5. Frees all dynamically allocated memory to ensure clean termination.

By passing all data through function arguments, the program avoids global variables and maintains modularity.

9.5. Error Handling

The solver is defensive in handling edge cases:

- Empty files or grids.
- Rows of inconsistent length.
- Invalid characters in the grid.
- Memory allocation failures.

In every case, resources are released properly and appropriate error codes are returned. This approach prevents memory leaks and ensures predictable behavior.

9.6. Design Considerations

The solver prioritizes simplicity and correctness over performance optimization. While the brute-force search is not efficient for extremely large grids, it guarantees correct results for all positions and directions. The double-pointer grid representation allows flexible memory allocation and direct access to individual cells.

The program also emphasizes readability and maintainability:

- All major tasks are encapsulated in separate functions.
- Helper functions handle common operations.
- Comments and clear structure guide the reader through the workflow.

9.7. Conclusion

This word search solver is a reliable, well-structured C implementation that demonstrates careful attention to file parsing, input normalization, memory management, and defensive programming. It serves as a solid foundation for enhancements such as graphical interfaces, automatic grid extraction, or integration with larger puzzle-solving pipelines. Despite its simplicity, it provides a full-featured, standalone console solution for typical word search grids.

```
// Example usage:
Point a = {0,0}, b = {0,0};
if (search_word(grid, rows, cols, word, &a, &b))
    printf("(%d,%d)(%d,%d)\n", a.x, a.y, b.x, b.y);
else
    printf("Not found\n");
```

10. Rebuilding the puzzle

A core component of the overall word search solving system is the ability to reconstruct a usable puzzle representation from segmented image data and recognized letters. This functionality is implemented in `rebuild_puzzle.c`, which serves as the bridge between raw image preprocessing, optical character recognition, and the automated solver. In essence, it transforms visual inputs into a structured digital grid that the solver can interpret, while also ensuring the output can be visualized with highlights.

10.1. Overall Functionality

The main function of this module is `solve_and_highlight`. Its responsibilities include:

1. Loading the region of interest (ROI) for both the puzzle grid and the word list.
2. Reading the letters from individual cell images using the letter prediction module.
3. Reconstructing the grid and word list in memory in a format suitable for the solver.
4. Invoking the solver to locate words within the grid.
5. Writing solver results to a text file and visually highlighting found words on a copy of the original puzzle image.

This modular design separates image preprocessing, recognition, solving, and visualization while providing a single entry point for orchestrating the entire puzzle-solving workflow.

10.2. Reading the Puzzle Grid

The puzzle grid is defined by its ROI, which is saved in `output/grid_roi.txt` during the segmentation stage. This file contains six integers: the top-left corner coordinates of the grid (gx, gy), the width and height of the grid (gw, gh), and the number of columns and rows. `solve_and_highlight` reads this information and uses it to structure a two-dimensional character array representing the puzzle.

For each grid cell, the function constructs a filename corresponding to a pre-extracted BMP image (e.g., `output/grid/02_01.bmp`), which contains the isolated letter. The letter prediction module (`predict_letter_from_file`) is invoked for each cell, and the returned character is converted to uppercase to ensure uniformity. This process continues row by row, forming a complete, structured in-memory grid that mirrors the original puzzle.

10.3. Reconstructing the Word List

Similar to the grid, the word list is reconstructed from pre-extracted character images located in `output/words/`. The ROI for the word list is stored in `output/wordlist_roi.txt` and includes coordinates, dimensions, and the total number of words.

For each word, characters are loaded sequentially from individual BMP files, and the OCR module predicts each letter. The process handles case normalization, converting lowercase letters to uppercase to match the solver's expectations. Words exceeding the maximum allowed length are truncated safely, preventing buffer overflows. By the end of this process, an array of word strings is reconstructed in memory, ready for the solver to process.

10.4. Solving the Puzzle

Once both the grid and word list are reconstructed, the module invokes the solver from `solver.h`. Each word is converted to uppercase and searched for

in the grid using a brute-force, eight-directional search algorithm. For each successfully located word, the solver returns the start and end coordinates. Results are simultaneously printed to the console for immediate feedback and written to a solver results file (`solver_results.txt`) for later processing by the highlight module.

The module handles not-found words gracefully, marking them with a `NOT_FOUND` flag in the results file. This ensures that downstream modules, including highlighting, can ignore unsuccessful matches without causing errors.

10.5. Visualizing the Solution

After the solver has produced results, `solve_and_highlight` uses the `highlight` module to overlay visual markers on a copy of the original puzzle image. This step translates grid coordinates into pixel positions based on the ROI and cell dimensions. Semi-transparent lines are drawn over successfully found words to indicate their location without obscuring the underlying letters. The highlighted image is then saved to a specified output path, providing an intuitive, user-friendly visualization of the solution.

This integration of solving and visualization ensures that the reconstructed puzzle is not just a backend data structure but a complete, interactive artifact that can be displayed, analyzed, or verified by a user.

10.6. Technical Considerations

Several design and implementation choices in `rebuild_puzzle.c` are critical for robustness and correctness:

- **Memory Safety:** All arrays are statically sized to prevent buffer overflows, and string operations use bounds-checked methods.
- **Error Handling:** Files are verified for existence and correct format before processing. Missing or malformed files trigger informative error messages.

- **Case Normalization:** Letter predictions are converted to uppercase to maintain consistency with the solver's search logic.
- **Modularity:** The function orchestrates multiple modules—OCR, solver, and highlight—without embedding their logic directly, allowing each module to remain independently testable.
- **Extensibility:** Constants such as `MAX_GRID_ROWS` and `MAX_WORDS` provide clear limits but can be adjusted to handle larger puzzles if needed.

10.7. Integration in the Processing Pipeline

The rebuild puzzle module acts as the central orchestrator connecting the earlier stages (image preprocessing, layout detection, cell extraction, and letter prediction) with the solver and final visualization. In the pipeline:

1. **Preprocessing and Segmentation:** Extracts individual cells and word images from the original puzzle.
2. **Letter Prediction:** Classifies the contents of each cell and word image into characters.
3. **Puzzle Reconstruction:** Combines predictions into a coherent grid and word list.
4. **Solver Execution:** Searches the reconstructed grid for all words in the list.
5. **Highlighting and Visualization:** Overlays solver results on the original image for intuitive presentation.

This structured flow allows the system to handle raw images of word search puzzles and automatically produce solved, annotated outputs.

10.8. Summary

The `rebuild_puzzle.c` module is the linchpin that enables the system to bridge the gap between image recognition and algorithmic solving. By carefully reconstructing both the puzzle grid and word list, orchestrating solver execution, and providing visual feedback, it ensures that the downstream MVP components operate on accurate, complete data. It encapsulates the complexity of integrating multiple low-level operations while presenting a simple, unified interface to higher-level modules, making the automated word search solver both functional and user-friendly. In short, it transforms segmented, preprocessed image data into a fully solved and visually annotated puzzle, providing the foundation upon which the GUI-based MVP is built.

11. Final minimal viable product (MVP)

The culmination of this project is a fully functional Minimal Viable Product (MVP) that integrates all previously developed components—image preprocessing, layout detection, cell extraction, word prediction, and solver logic—into a single interactive application. The MVP provides a graphical interface for users to input a puzzle image, process it, segment it, and solve the word search automatically, highlighting results in real time.

11.1. Overview of the Application Structure

The MVP is implemented using SDL2, SDL_image, and SDL_ttf for graphical interface management, image handling, and text rendering, respectively. The main components are:

- **GUI Layer:** Handles window creation, event polling, button rendering, and text input.
- **Image Management:** Maintains loaded and processed surfaces and textures, allowing users to view the current state of the puzzle.

- **Button Actions:** Each button invokes a specific stage of the pipeline—load, preprocess, denoise, segment, or solve.
- **Modular Backend:** All computational logic from previous programs (preprocessing, layout detection, cell and word extraction, solver) is wrapped in reusable functions.

This layered structure ensures separation of concerns: the GUI handles user interaction, while the backend performs all puzzle analysis.

11.2. Image Input and User Interaction

Users begin by providing an image path through a custom SDL text input popup. This component is implemented in the function `sdl_input_popup`, which opens a small window for typing, supporting basic editing and OK/Cancel buttons. Text rendering uses a TTF font to provide a readable, responsive interface. Input is validated, and empty or cancelled entries are ignored.

Once the image path is provided, the application loads the image using `IMG_Load` and stores it in `loaded_surface`. The window is resized dynamically to fit the image, ensuring a consistent visual layout. The original image is also saved to the output folder for reference.

11.3. Preprocessing Pipeline

The MVP provides two preprocessing options:

1. **Basic Preprocessing:** Converts the image to a binary or enhanced format without denoising. This is useful for images that are already clear.
2. **Preprocessing + Denoise:** Applies additional noise reduction to improve OCR and segmentation accuracy.

The preprocessing functions (`preprocess_no_denoise` and `preprocess`) produce intermediate surfaces that are scaled to fit the screen if necessary. The scaled or processed surface replaces any previously processed surface, and is saved as `processed.bmp` for later stages. This step ensures the solver operates on a standardized, noise-minimized image.

11.4. Segmentation and Cell Extraction

Segmentation is initiated via the `Segment` button. The function `detect_layout` analyzes the processed image to locate the overall puzzle area. Once the layout is identified, cells are extracted using `extract_cells`, and the list of words to be found is obtained via `extract_wordlist`. The resulting segmented image is stored as `stage_cells.bmp` and displayed to the user. This modular design allows each processing stage to be tested independently while providing visual feedback.

11.5. Solver Integration

The solver module, which was previously developed and verified as a standalone word search engine, is fully integrated. When the user clicks `Solve`, the function `solve_and_highlight` is called. It receives the processed and segmented image as input, runs the solver logic to locate all target words, and generates an output image with solved words highlighted. The results are also saved to a text file for record-keeping. If successful, the highlighted solved image is displayed in the main window, giving immediate visual confirmation.

11.6. Dynamic GUI Layout

The GUI dynamically arranges buttons and images to maximize usability. Button coordinates are recalculated each frame based on the current image dimensions. The rendering loop ensures both loaded and processed surfaces are displayed side by side or in layered form, with buttons drawn beneath

the images. SDL textures are created from surfaces for rendering efficiency, and all surfaces are managed carefully to avoid memory leaks.

11.7. Output and Resource Management

The MVP includes automated output folder management. The function `clear_output_directory` recursively deletes all contents of the output folder to avoid stale data from previous runs. Every loaded or processed surface is freed when no longer needed, and textures are destroyed to prevent GPU memory leaks. At application termination, SDL subsystems are properly shut down, ensuring a clean exit.

11.8. Summary of MVP Workflow

The final workflow of the MVP can be summarized as follows:

1. Load an image via file path input.
2. Apply preprocessing (with optional denoising).
3. Detect layout and extract individual cells.
4. Extract the word list for solving.
5. Run the solver and generate highlighted output.
6. Display each stage in the GUI for user feedback.

This workflow demonstrates how the previous modules—preprocessing, layout detection, cell and word extraction, and the solver—combine to form a single, usable application. Each stage is modular, allowing for future improvements or alternative algorithms to be plugged in without altering the overall structure.

11.9. Key Design Considerations

- **Modularity:** Each stage is implemented as a separate function or module, reflecting the earlier design of individual programs.
- **Robustness:** Defensive programming ensures that invalid inputs, missing files, or processing failures do not crash the application.
- **User Feedback:** Real-time display of loaded and processed images helps the user track progress.
- **Cross-Stage Integration:** Outputs from one module (e.g., segmented cells) seamlessly feed into the next stage (solver) without manual intervention.

The MVP represents a successful synthesis of all prior work into an interactive, practical tool, achieving the core goal of solving word search puzzles from arbitrary images with minimal user effort.

11.10. Highlighting Solved Words

The second crucial part of the MVP is the visualization of solver results on the original puzzle image. This functionality is encapsulated in the `highlight` module, implemented in `highlight.c`, which reads solver output and draws semi-transparent lines over found words. This step bridges the computational backend with the visual frontend, providing immediate, intuitive feedback to the user.

11.11. Algorithm Overview

The core routine, `highlight_words_on_image`, operates in the following sequence:

1. **Image Loading:** The original puzzle image is loaded into an SDL

surface using `IMG_Load`. This surface provides pixel-level access for drawing highlights.

2. **Solver Results Parsing:** The function reads a structured text file containing the positions of each found word. Each line typically contains the word and its start and end coordinates within the grid.

3. **Coordinate Transformation:** The grid-based coordinates of each word are converted to pixel positions. This accounts for the cell dimensions (width and height) relative to the original image, and also adds an offset if the puzzle does not occupy the entire image.

4. **Drawing Lines:** For each word, a thick line is drawn from the start cell to the end cell using the helper function `draw_thick_line_surface`. This function supports transparency and variable thickness, enabling visually clear highlighting without completely obscuring the underlying letters.

5. **Output Generation:** The modified image is saved as a PNG file, preserving both the original image content and the overlay of highlighted words.

11.12. Thick Line Rendering

The highlighting function implements a custom thick line renderer rather than relying on simple pixel manipulation. It calculates the vector between start and end points, computes perpendicular offsets for thickness, and iteratively draws blended pixels to achieve semi-transparency. Alpha blending is used to combine the highlight color with the original pixel color, ensuring that letters remain legible beneath the highlights. The default configuration uses semi-transparent red with a moderate thickness, providing high visibility without dominating the image.

11.13. Robustness and Safety

Several defensive measures ensure robustness:

- **Clamping Coordinates:** Both grid and pixel coordinates are clamped to the bounds of the image and the grid to prevent segmentation faults.
- **File Validation:** Missing or malformed solver result files are handled gracefully with error messages, preventing the program from crashing.
- **Surface Management:** The SDL surface is freed after the output image is saved, ensuring no memory leaks.

Empty or not-found lines in the solver results are ignored automatically, allowing the highlighting function to focus solely on successfully located words.

11.14. Integration with MVP Workflow

The highlight module is seamlessly integrated with the solver module in the final MVP. After the solver completes its computation and writes the results to `solver_results.txt`, the `solve_and_highlight` wrapper is invoked:

1. It reads the segmented or preprocessed image.
2. Runs the solver to locate all target words.
3. Calls the highlight module to overlay found words on the original image.

This ensures that the GUI can immediately display a visually annotated image to the user, maintaining the consistent workflow established in the MVP:

Load Image → Preprocess → Segment → Solve → Highlight → Display Results

The highlighting step transforms abstract solver output into a concrete, user-friendly visualization. Users can immediately see which words have been found, where they are located, and how the solver interpreted the puzzle grid. This feedback loop is essential for usability and provides an intuitive closure to the automated solving process.

11.15. Summary of Highlight Functionality

- Converts solver grid coordinates to precise pixel positions.
- Draws thick, semi-transparent lines over found words.
- Handles errors gracefully and ensures memory safety.
- Integrates directly with the solver module to provide immediate visual feedback.
- Completes the end-to-end workflow of the MVP, delivering a polished, user-facing result.

By combining the computational strength of the solver with the visual clarity of the highlight module, the MVP achieves its goal: a fully automated, interactive, and visually informative word search puzzle solver. Users can provide an image of a puzzle and observe the complete solution process from raw image to highlighted output without any manual intervention.

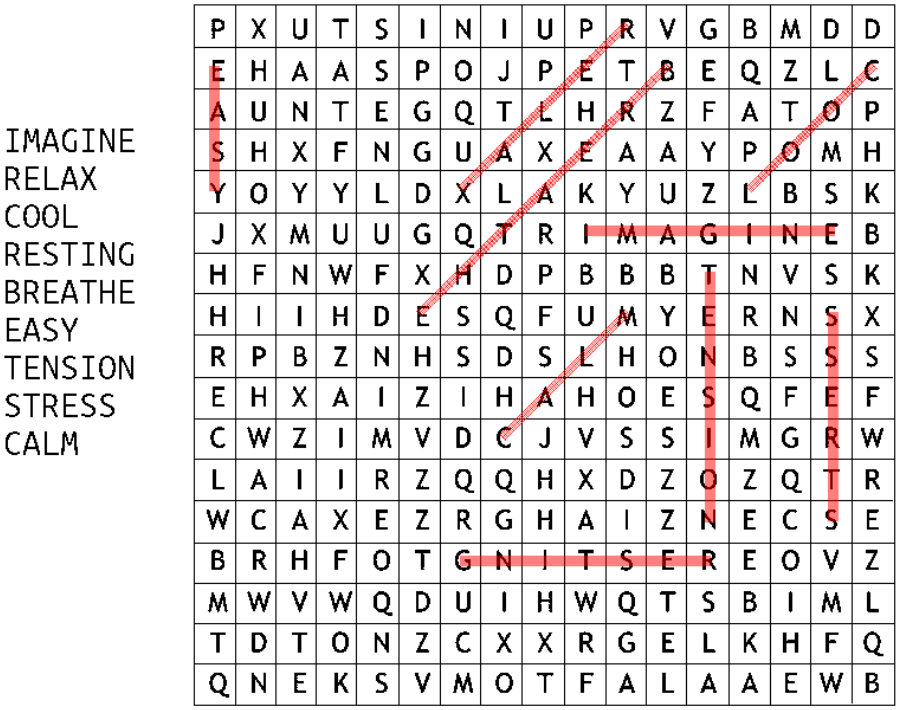

Figure 6: Final result.

12. Conclusion

This project presented the design and implementation of a fully functional automated word search solver, integrating multiple components into a cohesive software system. Beginning with robust image preprocessing, the program is capable of loading a puzzle image, applying denoising and binarization, and segmenting the image into individual cells and word lists. Advanced computer vision techniques were applied to detect the layout and extract each letter, leveraging a trained prediction model to ensure high accuracy in letter recognition.

The extracted letters and word lists are then reconstructed into a grid suitable for algorithmic processing. A modular solver, implemented in C, systematically searches for each word across all possible directions, employing precise memory management and thorough error handling. Once words are located, the solver generates results that are seamlessly mapped back onto the original image, highlighting found words in a clear and visually meaningful manner.

The integration of the highlight module ensures that users can immediately see the solution without manual cross-referencing, bridging the gap between computational results and visual interpretation.

The final Minimal Viable Product (MVP) demonstrates the successful combination of image processing, letter prediction, algorithmic word searching, and graphical highlighting into a single interactive application. The SDL-based graphical interface allows users to load images, preprocess, segment, and solve puzzles in a structured workflow, providing real-time feedback at each stage. Emphasis was placed on modularity, with clear boundaries between preprocessing, solving, and visualization components, allowing for future extension, such as integrating more sophisticated prediction models or alternative puzzle types.

In addition to technical functionality, the project highlights the importance of defensive programming, memory safety, and careful data handling. Each stage of the pipeline performs thorough validation, ensuring resilience against malformed inputs, missing files, or unexpected edge cases. This robustness, coupled with an intuitive interface, makes the application both reliable and user-friendly.

In conclusion, the project successfully transforms a complex, multi-step problem into a systematic, automated workflow. From raw image to fully highlighted solution, the system demonstrates an end-to-end approach that combines computer vision, algorithmic problem solving, and interactive visualization. The work establishes a solid foundation for further research and development in automated puzzle solving, with opportunities to improve accuracy, scalability, and user experience in future iterations.